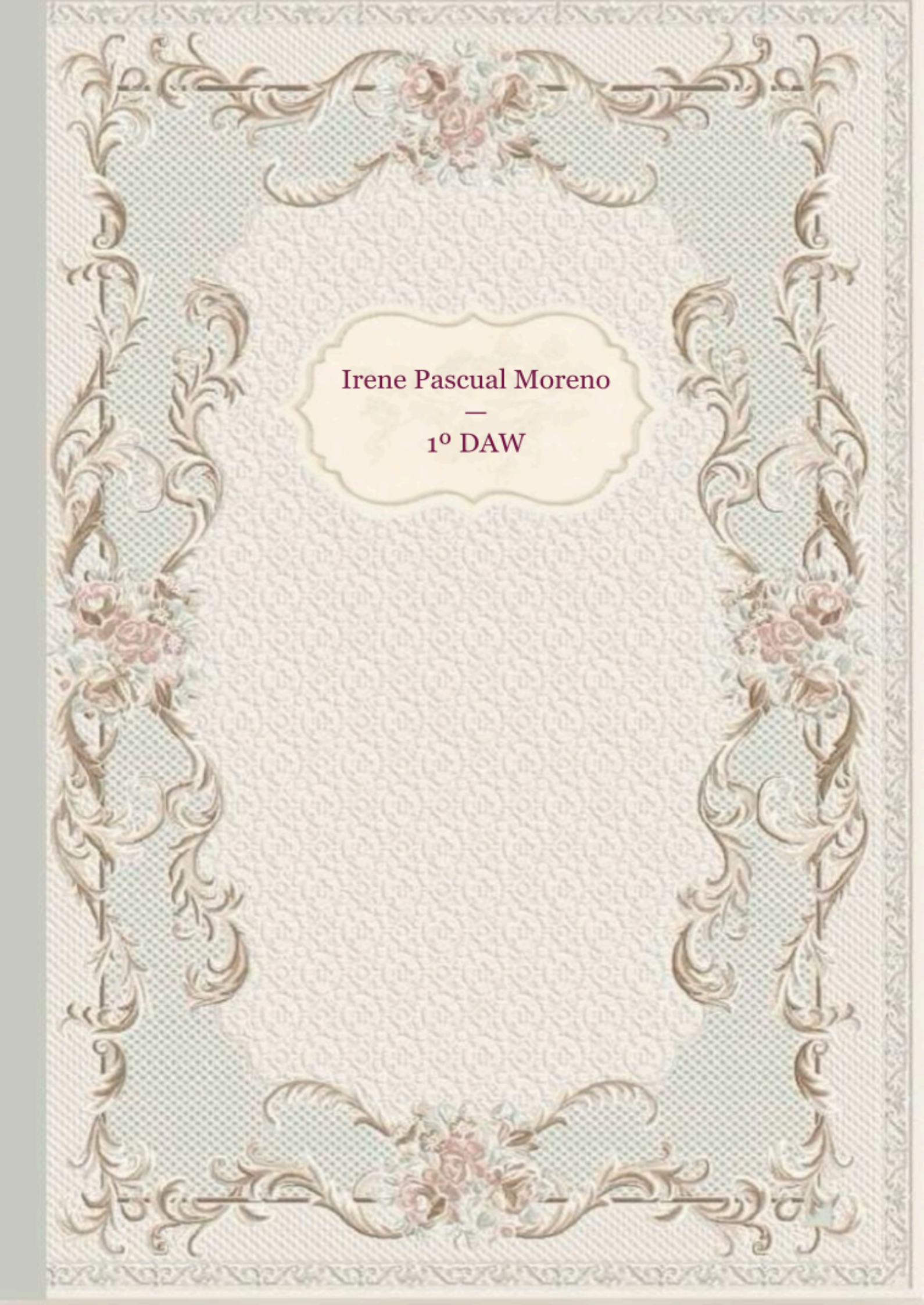




Irene Pascual Moreno

—
1º DAW

ÍNDICE

INTRODUCCIÓN.....	3
FASE 1: ANÁLISIS ESTÁTICO - SONARLINT.....	3
FASE 2: RED DE SEGURIDAD - JUNIT.....	4
FASE 3: REFACTORIZACIÓN CON EL IDE.....	5
3.1: Rename.....	5
3.2: Extract Method.....	6
3.3: Extract Constant.....	7
FASE 4: INTEGRACIÓN CONTINUA.....	7
ENLACE AL REPOSITORIO.....	8

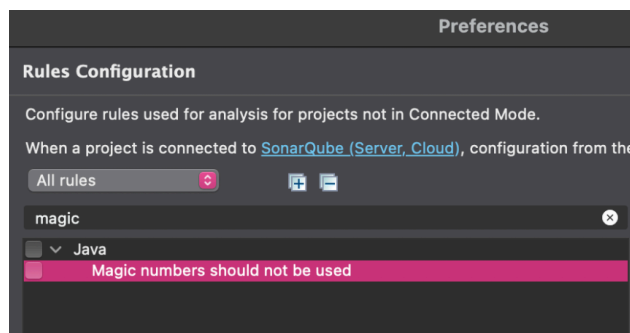
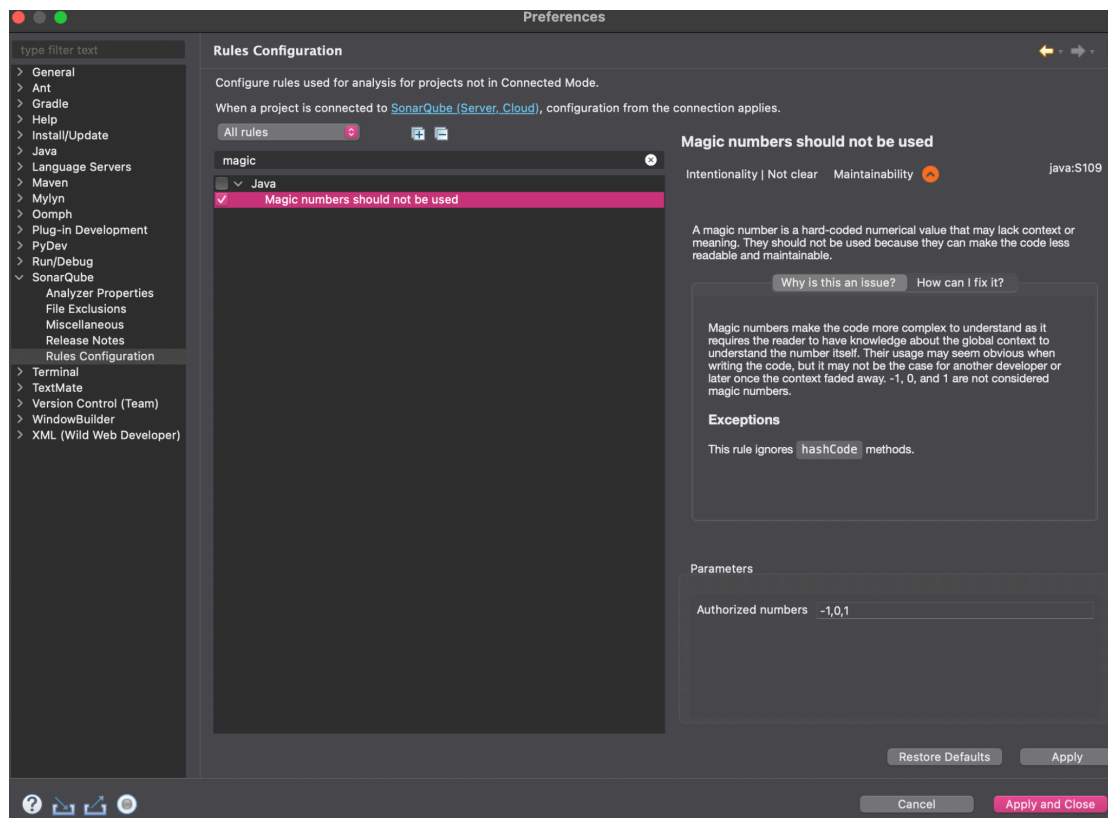
INTRODUCCIÓN

En esta práctica abordamos la mejora de un módulo Java mediante el uso de análisis estático, pruebas unitarias, refactorización controlada y automatización del proceso de validación.

FASE 1: ANÁLISIS ESTÁTICO - SONARLINT

El objetivo principal es mejorar la legibilidad, mantenibilidad y fiabilidad del código sin alterar su comportamiento funcional, aplicando buenas prácticas propias del desarrollo profesional.

Para el análisis estático del código he usado el plugin SonarQube for IDE (anteriormente SonarLint), que detecta problemas de mantenibilidad y calidad en tiempo real dentro del IDE.



En la versión local del plugin la severidad de determinadas reglas es fija, por lo que no es posible modificarla manualmente, pero he comprobado el impacto de la regla mediante su desactivación y posterior reactivación, cumpliendo el objetivo de análisis solicitado.

```
9
10 public class ProcesadorPedidos {
11
12
13     public double procesar(ArrayList<String> a, ArrayList<Double> b) {
14         double t = 0;
15
16         // Sumar precios de la lista
17         for (int i = 0; i < b.size(); i++) {
18             System.out.println("Añadiendo producto: " + a.get(i));
19             t = t + b.get(i);
20         }
21
22         // Lógica de descuento (Magic Number 100 y 0.10)
23         if (t > 100) {
24             System.out.println("Descuento aplicado.");
25             t = t - (t * 0.10);
26         }
27     }
28 }
```

Durante el análisis estático del código, gracias al plugin, he visto *code smells* adicionales, además de los números mágicos abordados durante la refactorización.

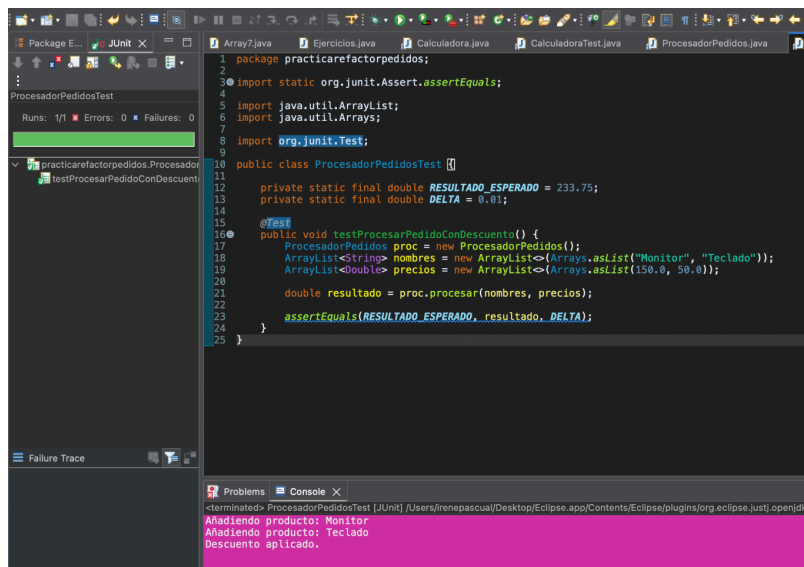
SonarQube for IDE sugiere el uso de interfaces (List) en lugar de implementaciones concretas (ArrayList) en las firmas de los métodos, con el objetivo de mejorar la flexibilidad y el desacoplamiento.

Después, la herramienta señala el uso de System.out.println como mala praxis en entornos productivos, recomendando el uso de sistemas de *logging*.

FASE 2: RED DE SEGURIDAD - JUNIT

Antes de realizar cualquier refactorización he implementado una prueba unitaria con JUnit 4 que valida el comportamiento original del método procesar.

Esta prueba se ejecuta tras cada cambio para asegurar que el comportamiento funcional del programa no se vea alterado.



He optado por sustituir los valores numéricos utilizados en la aserción (233.75 y 0.01) por constantes con nombre semántico.

He tomado esta decisión por dos motivos: Primero, mejora la legibilidad del test porque permite comprender de forma inmediata el significado de cada valor sin necesidad de interpretar cálculos implícitos. Y segundo, elimina la presencia de *números mágicos* dentro del código de prueba, alineando el test con las mismas buenas prácticas aplicadas posteriormente durante la refactorización del código productivo.

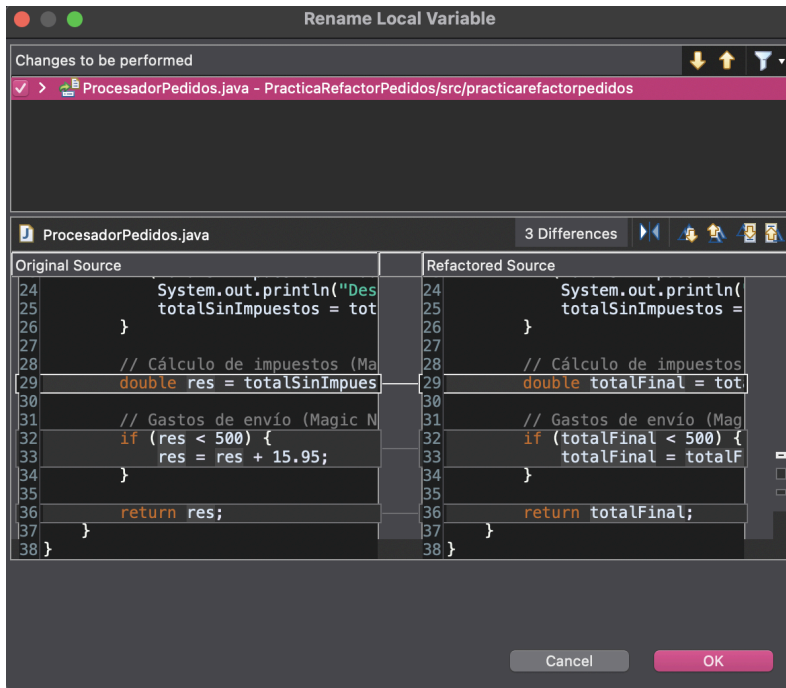
Además el uso de constantes facilita la modificación futura del test en caso de que cambien las reglas de negocio, evitando la dispersión de valores literales.

FASE 3: REFACTORIZACIÓN CON EL IDE

ACCIÓN EN ECLIPSE	MENÚ	ATAJO (Mac)
Rename	Refactor → Rename	⌘⇧R
Extract Method	Refactor → Extract Method	⌘⇧M
Extract Constant	Refactor → Extract Constant	No tiene atajo por defecto

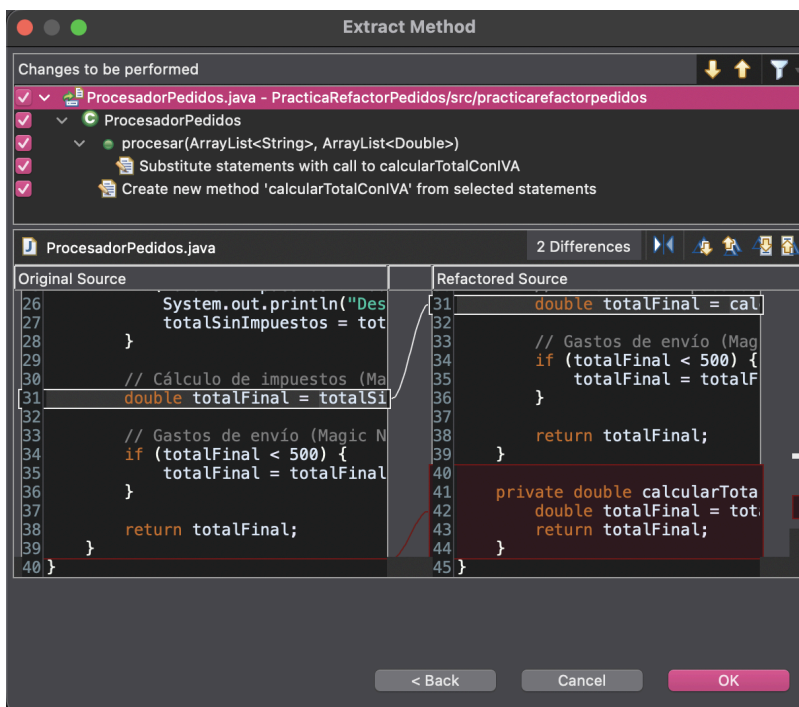
3.1: Rename

He renombrado variables con nombres genéricos por otros más significativos para mejorar la legibilidad y comprensión del código.



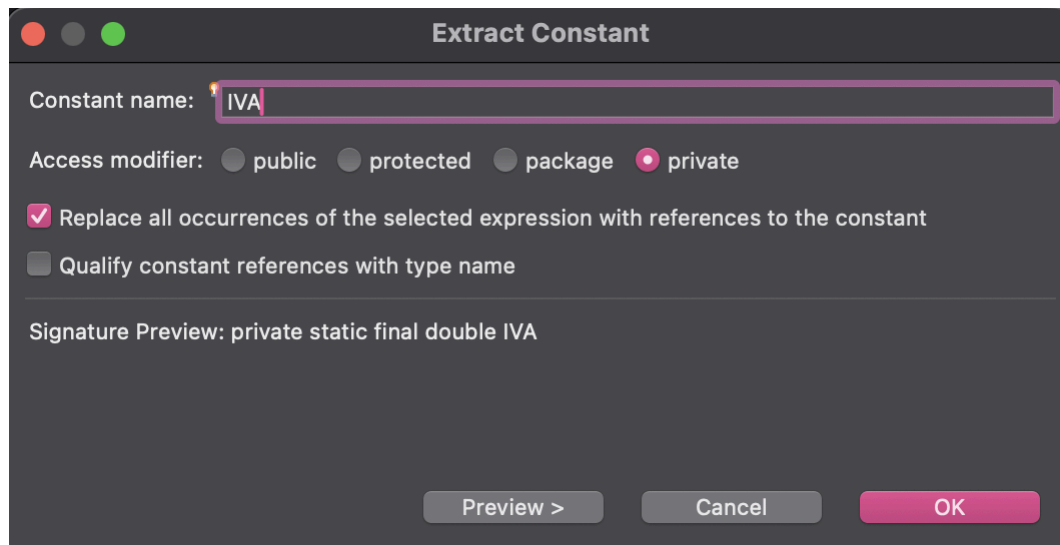
3.2: Extract Method

La lógica de cálculo del IVA y de los gastos de envío se ha extraído a métodos privados independientes, reduciendo la complejidad del método principal y mejorando la cohesión.

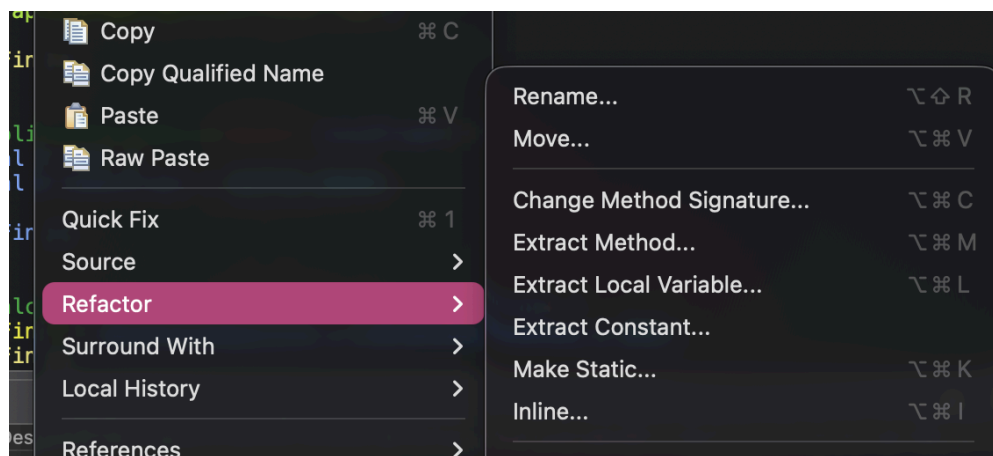


3.3: Extract Constant

Los valores numéricos sin contexto han sido sustituidos por constantes con nombres semánticos, facilitando el mantenimiento y posibles cambios futuros.



En Eclipse para macOS, la refactorización *Extract Constant* no dispone de un atajo de teclado asignado por defecto, por lo que se accede a ella mediante el Refactor.



FASE 4: INTEGRACIÓN CONTINUA

He configurado un flujo de Integración Continua mediante GitHub Actions que compila el proyecto y ejecuta los tests automáticamente en cada push.

Para verificar el correcto funcionamiento de la Integración Continua, he provocado intencionadamente el fallo de un test unitario modificando el valor esperado, así comprobamos que GitHub Actions detecta el error y marca el workflow como fallido. Una vez hecho esto, he restaurado el valor correcto, confirmando que el pipeline vuelve a

ejecutarse con éxito. Este proceso demuestra que la CI está configurada y funciona correctamente ante errores reales.

All workflows

Showing runs from all workflows

Filter workflow runs

3 workflow runs		Event ▾	Status ▾	Branch ▾	Actor ▾
✓	fix: restore correct expected value to pass CI Integracion Continua Java #3: Commit 2788847 pushed by irenepm777			main	1 minute ago 20s
✗	test: break CI with incorrect expected value Integracion Continua Java #2: Commit 662547b pushed by irenepm777			main	3 minutes ago 13s
✓	ci: add GitHub Actions workflow for Maven tests Integracion Continua Java #1: Commit 64ffb26 pushed by irenepm777			main	15 minutes ago 20s

ENLACE AL REPOSITORIO

Repositorio GitHub:

<https://github.com/irenepm777/practica-refactor-pedidos>